

TraderBot

Architecture, File System, and Operations Manual

elizaOS Solana memecoin trading bot deployed to AWS Frankfurt EC2.

Document generated for cross-AI handoff so a successor agent can pick up the workflow without context loss.

Generated: 2026-05-03 09:16 UTC

Repository: /home/paulf/eliza

Git branch: fix/ignore-action-upstream

Bot host: ubuntu@63.180.211.88 (Frankfurt EC2, Elastic IP)

Dashboard: <http://63.180.211.88:3001/>

1. Top-Level Overview

TraderBot is a Python-based Solana memecoin trading bot built on top of the elizaOS agent runtime. It monitors pump.fun bonding-curve token launches, evaluates them with multi-stage gates, executes buys via PumpPortal, and manages exits through a 3-tier LLM brain cascade (Groq, Gemini, Claude). All live state and metrics are exposed through an aiohttp HTTP server on port 3001 which also serves a React/TypeScript dashboard.

Repository layout (top-level)

```
/home/paulf/eliza/                                <- monorepo root (bun + Turbo)
+-- packages/
|   +-- typescript/                                @elizaos/core (authoritative TS runtime)
|   +-- python/                                    elizaos Python mirror runtime + plugins
|   |   +-- elizaos/
|   |   |   +-- runtime.py
|   |   |   +-- bootstrap/
|   |   |   +-- plugins/
|   |   |   |   +-- solana/    <- ALL TRADING LOGIC LIVES HERE
|   |   |   |   \__ openai/
|   |   |   \__ types/generated/ (protobuf-generated)
|   |   \__ run_traderbot.py    <- entry point (systemd ExecStart)
|   +-- dashboard/              React + Vite frontend (served by bot at :3001)
|   +-- schemas/                protobuf .proto files
|   +-- elizaos/                elizaos CLI binary
|   +-- interop/, daemon/, prompts/
+-- .env                        secrets (HELIUS, GROQ, GEMINI, ANTHROPIC, ...)
+-- .venv_py/                  Python virtualenv (bot uses this interpreter)
+-- CLAUDE.md                  project instructions for Claude Code
\__ creator_alpha_playbook.json + many other state files (in solana/ dir)
```

Polyglot design

The repository hosts the same agent model in three languages (TypeScript, Python, partial Rust). The TypeScript package in `packages/typescript/` is the authoritative reference. The bot runs the Python mirror (`packages/python/elizaos/runtime.py`) because most trading-bot work and the LLM brain integrations live in Python. The two runtimes share types via protobuf generation in `packages/schemas/` -> `bun run generate:types` regenerates both sides.

2. Bot Entry Point and Service Lifecycle

Entry point: packages/python/run_traderbot.py

This file is what systemd starts. It loads .env from repo root, instantiates AgentRuntime, registers the bootstrap plugin and the Solana plugin, then starts an aiohttp HTTP bridge on port 3001 for the dashboard, the Jarvis chat command centre, and the WebSocket feed.

```
load_dotenv(<repo_root>/.env)
runtime = AgentRuntime(...)
runtime.register_plugin(create_bootstrap_plugin())
runtime.register_plugin(create_openai_plugin())
runtime.register_plugin(create_solana_plugin())
await start_http_bridge(runtime) # aiohttp server :3001
await runtime.start()           # services boot here
```

packages/python/elizaos/plugins/solana/__init__.py

The Solana plugin factory wires together services, providers, and actions:

```
Plugin(
    name='solana',
    services=[
        SolanaWalletService,      # signs txs with the bot's keypair
        PumpFunService,           # pump.fun bonding curve helpers
        RaydiumService,           # raydium / pumpswap swap helpers
        TokenLaunchMonitorService, # WebSocket listener (legacy)
        SocialMonitorService,      # twitter/grok/telegram alpha
        PositionManagerService,    # tracks open positions, P&L
        MonsterStrategyService,    # the SCOUT ENGINE - launches all loops
    ],
    providers=[wallet_provider, market_provider, launches_provider],
    actions=[buy_token_action, sell_token_action, get_price_action],
)
```

MonsterStrategyService - the scout dispatcher

File: packages/python/elizaos/plugins/solana/services/monster_strategy.py

On start() it spawns these asyncio tasks (each a long-running scout loop in monster_signals.py):

- cluster_confirm_scout_loop - wallet-cluster confirmation entries (currently OFF)
- serial_deployer_scout_loop - watches known serial deployers (OFF)
- lifecycle_scout_loop - DexScreener-driven post-graduation entries (OFF)
- breakout_candle_scout_loop - candle-shape breakouts (OFF)
- creator_alpha_scout_loop - PRIMARY ACTIVE STRATEGY: polls direct creators + operators

Disabled scouts have a flag near the top of monster_signals.py (e.g. CLUSTER_CONFIRM_ENABLED = 'false', LIFECYCLE_Scout_ENABLED = False). Toggling those constants and redeploying re-enables the loop. Currently the only ACTIVE entry path is creator_alpha.

3. Strategy Files - Where to Make Changes

Detection and gating: `monster_signals.py`

Path: `packages/python/elizaos/plugins/solana/monster_signals.py`

This is the largest and most-edited file. It contains every scout loop, the gate functions that decide whether a candidate token gets bought, the operator/creator polling logic, the lifecycle-rejection book-keeping, and the priority-mint mechanism that lets creator_alpha jump the queue.

Key constants and flags at the top of the file

```
PUMP_FUN_PROGRAM = '6EF8rrecthR5Dkzon8Nwu78hRvfCKubJ14M5uBEwF6P'
HELIUS_RPC = 'https://mainnet.helius-rpc.com/?api-key=...'
CLUSTER_CONFIRM_ENABLED = 'false'
SERIAL_DEPLOYER_ENABLED = 'false'
LIFECYCLE_SCOUT_ENABLED = False
LIFECYCLE_VIRAL_ENABLED = False
BREAKOUT_SCOUT_ENABLED = 'false'
_CREATOR_ALPHA_MIN_FUND_SOL = 0.5
_CREATOR_ALPHA_MAX_FUND_SOL = 2.0
# The above two control the operator-funds-fresh-wallet detection band.
```

Key functions in `monster_signals.py`

- `creator_alpha_scout_loop(runtime, session)` - main loop, 60s cycle.
- `_creator_alpha_detect_create(session, sig, signer)` - returns mint if signer is recorded as create signer in a pump.fun tx.
- `_creator_alpha_detect_funding(session, sig, operator)` - returns (recipient, amount_sol) tuples within MIN/MAX SOL band.
- `_creator_alpha_direct_entry(runtime, mint, source)` - queues an entry that bypasses lifecycle gates.
- `mark_creator_alpha_priority(mint)` - adds mint to the priority set so other gates relax for it.
- Lifecycle-related: `lifecycle_evaluate_token()`, `record_lifecycle_reject()`, `record_lifecycle_pass()`.

Position management and exits: `strategy_e_monster.py`

Path: `packages/python/elizaos/plugins/solana/strategy_e_monster.py`

Responsible for every open position: sizing, opening trade via PumpPortal, the exit decision tree, and post-exit book-keeping.

Source-aware getter pattern (recently added)

The file now branches on `signal_source` so creator_alpha positions use different rails than standard monster positions. To change rails, edit `live_config` defaults (next section) AND the helper getters:

```
def _is_creator_alpha_source(source):
    return bool(source) and source.startswith('creator_alpha')

def get_size_for_source(source)          # returns 0.10 for creator_alpha, 0.45 default
def get_floor_for_source(source)         # returns -75.0 for creator_alpha, -25 default
def get_tp1_mult_for_source(source)      # returns 2.0 (+100%) for creator_alpha, 1.20 default
def get_tp1_sell_frac_for_source(source) # returns 0.5 for creator_alpha, 1.0 default
def can_open_new_position(source)        # separate slot pool per signal_source
```

Open / evaluate functions

- `open_monster_position(runtime, mint, sol_size, signal_source, source_metadata, pool=None)` - resolves entry price, signs+sends tx, registers in PositionManager.
- `evaluate_exit(runtime, position)` - called periodically; consults the brain decision and the source-aware floors / take-profits.
- Bonding-curve entry_price fallback: when DexScreener has not indexed the new mint, computes `entry_price = sol_size / (tokens_received_raw / 1_000_000)`.

Brain (LLM) decisions: `trade_monitor.py`

Path: `packages/python/elizaos/plugins/solana/trade_monitor.py`

Per-position 4-tier cascade. For each open position, three brains poll on different cadences:

- Groq llama-3.3-70b-versatile - 30s cycle, gut-check; primary exit signal from t=0.
- Gemini 2.5-flash (fallback 2.5-flash-lite on 5xx/429) - 2 min cycle, starting at t=60s.
- Claude Opus 4.7 (claude-opus-4-7) - 3 min cycle, starting at t=60s + emergency tier.

The shared prompt template `_HOLD_SELL_PROMPT` begins around line 1495. It now contains a source-aware ENVIRONMENT block: when `signal_source` starts with 'creator_alpha' the brains are told the rails are +100% TP1 / -75% catastrophic floor / brain-managed moonbag. Otherwise they get the standard +20%/-25% rails. The brain context dict already includes `signal_source`, `tp1_target_pct`, `floor_pct`, `moonbag_managed`.

Live config (RUNTIME source of truth): `live_config.py`

Path: `packages/python/elizaos/plugins/solana/live_config.py`

Persisted JSON: `packages/python/elizaos/plugins/solana/live_config.json` (read at startup, rewritten on each PATCH /api/config request).

IMPORTANT: `live_config` is the source of truth at runtime, NOT `.env`. `.env` is only used to seed defaults the first time `live_config.json` is created. To change a parameter on a running bot you either (a) PATCH /api/config from the dashboard, or (b) edit `live_config.json` on the server then restart, or (c) edit `live_config.py` defaults + delete `live_config.json` + restart.

Creator-alpha keys (the active strategy)

<code>creator_alpha_size_sol</code>	0.10	# SOL per entry
<code>creator_alpha_max_concurrent</code>	3	# parallel positions
<code>creator_alpha_floor_pct</code>	-75.0	# catastrophic floor (no tight SL)
<code>creator_alpha_tp1_mult</code>	2.0	# +100% TP1 multiplier
<code>creator_alpha_tp1_sell_frac</code>	0.5	# sell 50% at TP1, keep 50% as moonbag

Standard monster keys (legacy, mostly unused now)

<code>monster_max_concurrent</code>	1	
<code>monster_default_size_sol</code>	0.45	
<code>monster_strategy_enabled</code>	true	
<code>monster_tp1_gain_pct</code>	20	# +20% full exit
<code>monster_tp1_sell_fraction</code>	1.0	
<code>monster_pre_tp1_floor_pct</code>	-25	

4. Dashboard - React/TypeScript Frontend

Layout overview

Path: packages/dashboard/

Stack: React 18 + Vite + TypeScript. In dev (bun run dev) Vite proxies /api, /ws, /message, /health to localhost:3001 (the bot). In production the bot serves the static dist/ directory directly via aiohttp's add_static at /assets and a SPA-fallback handler.

Component directory structure

```
packages/dashboard/src/
+-- App.tsx                <- root, hooks WebSocket + status, mounts panels
+-- api/client.ts          <- thin fetch wrappers; baseUrl is '' (same-origin)
+-- store/dashboard.ts     <- zustand global state (positions, equity, etc.)
+-- hooks/useWebSocket.ts  <- WS to /ws for live broadcasts
\__ components/
  +-- activity/            ActivityFeed (event stream)
  +-- brains/              BrainPanel (per-position brain decisions)
  +-- chart/               price/equity charts
  +-- chat/                BotChat (Jarvis chat console)
  +-- common/              shared UI primitives
  +-- copytrade/           CopyTradeHistoryTable, CopyTradeLiveCharts (legacy, mostly hidden)
  +-- diagnostics/        <- WHERE STRATEGY-SPECIFIC PANELS LIVE
    | CreatorAlphaPanel.tsx - playbook + activity
    | CreatorAlphaPnLPanel.tsx - source-filtered P&L for creator_alpha
    | LifecycleRejectsPanel.tsx - what got blocked and why
    | PositionConfigPanel.tsx - current live_config knobs
    | PerformanceBySourcePanel.tsx - WR/PnL grouped by signal_source
  +-- history/             trade history table
  +-- launches/            pump.fun launch feed
  +-- layout/              AlphaDashboardLayout (top-level grid)
  +-- portfolio/           wallet balance + equity
  +-- positions/           PositionsTable, CopyTradePositions
  +-- risk/                risk gauges
  \__ wallets/            watched-wallet list
```

Dashboard data flow

1. App.tsx mounts and calls fetchStatus() (one-shot GET /api/status) for initial state.
2. useWebSocket() opens /ws; the bot's _broadcast_loop pushes positions/equity/activity events.
3. Each diagnostic panel does its own polling fetch (e.g. CreatorAlphaPanel hits /api/creator-alpha every ~10s) - look at the panel's useEffect to find its endpoint.

Adding or editing a panel

1. Edit or create the .tsx file under components/<section>/.
2. If new, add an import + JSX usage in App.tsx (or in the relevant layout file).
3. If the panel needs new data, add a route in dashboard_api.py (see next section).
4. Build the dashboard: cd packages/dashboard && bun run build (output goes to dist/).
5. SCP the dist/ to AWS (see Deployment section).

5. HTTP API - dashboard_api.py

Path: packages/python/elizaos/plugins/solana/dashboard_api.py

All HTTP routes are registered here on the same aiohttp Application instance the run_traderbot.py bridge created. Entry point: register_dashboard_routes(app, runtime). The middleware adds CORS for dev.

Route catalogue (current)

GET	/api/status	system + position summary (one-shot)
GET	/api/positions	open positions array
GET	/api/history?limit&offset	closed trades
GET	/api/equity?since	equity timeseries
GET	/api/launches?limit	recent pump.fun launches
GET	/api/report	daily debrief report
POST	/api/control	pause / resume / kill switches
PATCH	/api/config	update any live_config key (validated)
POST	/api/positions/{mint}/close	manually close a position
GET	/api/whitelist	creator/dev whitelist
POST	/api/whitelist	edit whitelist
POST	/api/speak	TTS endpoint
POST	/api/chat	Jarvis chat fallback
GET	/api/copy-trade/stats	legacy copy-trade metrics
GET	/api/frost-mirror/stats	legacy frost mirror
GET	/api/lifecycle/rejects	what was rejected and why
GET	/api/creator-alpha	playbook + recent activity
GET	/api/performance-by-source	WR/PnL grouped by signal_source
GET	/api/holder-guard/report	holder distribution analysis
GET	/api/near-miss-report	near-miss tracker output
POST	/api/copy-trade/close/{mint}	
POST	/api/copy-trade/pause	
GET	/api/wallet-promotion	wallet promotion / demotion log
GET	/api/brain-memory	brain memory (claude/gemini/groq) snapshot
POST	/api/webhooks/helius/pumpswap-grad	Helius webhook for graduations
GET	/ws	WebSocket broadcast feed
GET	/assets/* + /{path:.*)	static SPA serving (production only)

Adding a new route

Open dashboard_api.py, scroll to the section near the bottom where app.router.add_get(...) calls are clustered. Define an async handler `async def handle_foo(request) -> web.Response:` that reads runtime state via the closed-over `runtime` reference and returns `web.json_response(...)`. Then add `app.router.add_get('/api/foo', handle_foo)` next to the others. No registration boilerplate needed - just edit, deploy, restart.

6. State and Data Files (in solana/ directory)

These JSON files are read/written by the running bot. Path prefix: packages/python/elizaos/plugins/solana/

Curated lists (you edit these by hand or via dashboard)

- creator_alpha_playbook.json - tiers of direct_creators (tier_a/b/c, high_hit) and operators. The creator_alpha scout polls every wallet listed here on a 60s cycle. Adding a wallet = add its address with metadata, redeploy.
- creator_whitelist.json + monster_creator_whitelist.json - allow-list of creator wallets.
- dev_lists.json + smart_money_list.json + smart_wallets.json - other curated rosters.
- monster_addresses.json + monster_creator_intel.json - reference data for the monster strategy.

Live operational state (updated by the bot)

- trade_history.json - closed trades. Wipe on session reset.
- traded_mints.json - permanent blacklist. NEVER WIPE.
- cooldowns.json - per-wallet/strategy timeouts. Safe to wipe.
- creator_alpha_candidates.json - live pending mints from the scout.
- learning_engine outcomes -> learning_outcomes.json (in /home/ubuntu/eliza on AWS).
- rejected_tokens.json + near_misses.json - rejection book-keeping.
- post_exit_tracker.json - tracks what the price did AFTER we exited (for learning).
- strategy_e_signals.json - monster-strategy candidate log.
- brain memory: claude_brain_memory.json, gemini_brain_memory.json, groq_brain_memory.json - per-mint context the brains can recall across cycles.
- jarvis_conv_history.json + jarvis_winning_configs.json - Jarvis chat memory.
- trading_journal/ - per-day journal entries split into _live.json / _paper.json.

Reset recipes

```
# Fresh trade-history reset (DOES NOT touch permanent blacklist):
echo '[]' > packages/python/elizaos/plugins/solana/trade_history.json
echo '{} ' > packages/python/elizaos/plugins/solana/cooldowns.json
# Then restart bot.

# DO NOT run: rm traded_mints.json <- destroys permanent blacklist.
```

7. Secrets and .env

Path: /home/paulf/eliza/.env (mirrored on AWS at /home/ubuntu/eliza/.env)

Loaded by run_traderbot.py via dotenv. The bot then PROMOTES a subset of these to live_config (creator_alpha rails, monster sizing, etc.) which becomes the actual runtime source of truth. Keys you typically care about:

```
HELIUS_API_KEY=...      # Helius RPC + LaserStream
GROQ_API_KEY=...        # llama-3.3-70b-versatile brain
GEMINI_API_KEY=...      # gemini-2.5-flash brain
ANTHROPIC_API_KEY=...   # claude-opus-4-7 brain
PUMPPORTAL_API_KEY=...  # only required for pumpportal-lightning, NOT for trade-local
TELEGRAM_BOT_TOKEN=...  # Telegram alerts
WALLET_PRIVATE_KEY=...  # Solana keypair (base58)
MONSTER_STRATEGY_ENABLED=true
COPY_TRADE_ENABLED=false
```


8. Deployment - Local -> Git -> AWS

AWS infrastructure

Provider	AWS EC2, Frankfurt region (eu-central-1)
Elastic IP	63.180.211.88 (permanent)
SSH user	ubuntu
SSH key	/home/paulf/.ssh/traderbot-frankfurt-key.pem
Project root on EC2	/home/ubuntu/eliza/
Python venv on EC2	/home/ubuntu/eliza/.venv_py/
systemd unit	/etc/systemd/system/traderbot.service
Service log file	/home/ubuntu/eliza/traderbot.out (stdout + stderr appended)
Dashboard URL	http://63.180.211.88:3001/
Listening port	3001 (aiohttp serves both API and SPA)

systemd unit (verbatim from the box)

```
[Unit]
Description=TraderBot Solana Trading Bot
After=network-online.target
Wants=network-online.target

[Service]
Type=notify
User=ubuntu
WorkingDirectory=/home/ubuntu/eliza
ExecStartPre=/bin/bash -c "fuser -k 3001/tcp || true"
ExecStart=/home/ubuntu/eliza/.venv_py/bin/python packages/python/run_traderbot.py
Restart=always
StartLimitBurst=0
WatchdogSec=120
StandardOutput=append:/home/ubuntu/eliza/traderbot.out
StandardError=append:/home/ubuntu/eliza/traderbot.out

[Install]
WantedBy=multi-user.target
```

Notes: Type=notify means the service announces readiness via sd_notify (the Python code calls systemd-notify on startup). WatchdogSec=120 -> the bot must call notify("WATCHDOG=1") within 120s or systemd kills+restarts it. ExecStartPre kills anything still bound to 3001 from the previous run.

Standard deployment workflow (Python-only changes)

```
# 1. Edit files locally in /home/paulf/eliza/...
# 2. Verify Python syntax (optional but recommended):
/home/paulf/eliza/.venv_py/bin/python -c "import ast; ast.parse(open('<path>.py').read())"

# 3. Commit (only when user explicitly asks):
git add <files>
git commit -m "<concise message>"
# Convention: lowercase scope: imperative verb -- short rationale
# Examples from recent log:
#   playbook: add 4euyaqBe as MOG dev wallet
#   monster: fix stale-create bug + entry_price BC fallback
#   brains: source-aware HOLD/SELL prompt -- creator_alpha rails
```

```
# 4. SCP changed files to AWS:
scp -i /home/paulf/.ssh/traderbot-frankfurt-key.pem \
    packages/python/elizaos/plugins/solana/<file>.py \
    ubuntu@63.180.211.88:/home/ubuntu/eliza/packages/python/elizaos/plugins/solana/<file>.py

# 5. Restart the service so the new code is picked up:
ssh -i /home/paulf/.ssh/traderbot-frankfurt-key.pem ubuntu@63.180.211.88 \
    "sudo systemctl restart traderbot.service && sleep 3 && \
    sudo systemctl is-active traderbot.service"

# Expect output: active
```

JSON-only changes (e.g. playbook updates)

Same pipeline but only the .json file gets SCPed. Always validate JSON locally before deploying:

```
python3 -c "import json; json.load(open('<file>.json')); print('valid')"
```

Dashboard (TypeScript) changes

```
# 1. Edit packages/dashboard/src/...
# 2. Build:
cd packages/dashboard && bun run build
# Output: packages/dashboard/dist/ (assets/ + index.html)

# 3. SCP whole dist directory:
scp -i /home/paulf/.ssh/traderbot-frankfurt-key.pem -r \
    packages/dashboard/dist/ \
    ubuntu@63.180.211.88:/home/ubuntu/eliza/packages/dashboard/

# 4. Bot restart NOT strictly required for static asset changes -
# the aiohttp static handler reads from disk on each request - but
# restarting is the safe default if any backend route also changed.
```

Tail the bot log to verify

```
ssh -i /home/paulf/.ssh/traderbot-frankfurt-key.pem ubuntu@63.180.211.88 \
    "tail -f /home/ubuntu/eliza/traderbot.out"

# Useful greps:
ssh ... "grep -E 'creator-alpha|MONSTER|ERROR|trace' /home/ubuntu/eliza/traderbot.out | tail -50"
```

CI / GitHub Actions

There is a workflow at [.github/workflows/deploy-dashboard.yml](#) (currently untracked). It is NOT in the standard deploy path used today - the human-in-the-loop scp + systemctl restart described above is the canonical deployment method. CI may be wired up in future.

9. Git Workflow

Branching

Working branch: fix/ignore-action-upstream (this is the long-lived integration branch where most work happens). Main branch: main. New features and fixes are pushed straight to the working branch. PRs are not part of the routine flow at the moment - the user reviews diffs locally.

Commit conventions

- Lowercase scope prefix: 'monster:', 'brains:', 'playbook:', 'dashboard:', 'jarvis:'.
- Imperative verb: 'add', 'fix', 'revert', 'tighten', 'deploy'.
- Short rationale after a single em-dash where helpful.
- Recent example: 'monster: fix stale-create bug + entry_price bonding-curve fallback'.
- Co-author footer: include 'Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>' when Claude produced the change.

CRITICAL git rules (from CLAUDE.md)

- NEVER commit unless the user explicitly asks.
- NEVER use git -i flags (no interactive rebase / add).
- NEVER force-push to main.
- Prefer NEW commits over --amend.
- Add specific files, not 'git add -A' (avoids leaking .env / large binaries).

10. AI Brain Architecture (trade_monitor.py)

Each open position spawns three async tasks in trade_monitor.py - one per LLM provider. They share a context dictionary built from the latest position metrics, recent price action, holder deltas, and the source-aware ENVIRONMENT block.

Tier cadences

Groq tier	llama-3.3-70b-versatile, every 30s from t=0. Primary exit signal.
Gemini tier	gemini-2.5-flash (fallback flash-lite on 429/5xx), every 2 min from t=60s.
Claude tier	claude-opus-4-7, every 3 min from t=60s + emergency tier on rapid drawdown.

Brain context dictionary (selected fields)

```
{
  'mint': '<address>',
  'signal_source': 'creator_alpha' | 'monster_lifecycle' | 'copy_trade_*' | ...,
  'tp1_target_pct': 100.0 | 20.0,
  'floor_pct': -75.0 | -25.0,
  'moonbag_managed': true | false,
  'pnl_pct': ...,
  'holders_now': ...,
  'holder_delta_5min': ...,
  'liq_chg_5min_pct': ...,
  'volume_h1': ...,
  'entry_verdict': 'RUNNER' | 'NORMAL' | 'RUG_RISK',
  'whale_pattern': '...',
  'pattern_rules': [<learned rule strings>],
  'recent_trades': [...],
}
```

HOLD / SELL prompt

Defined as the constant _HOLD_SELL_PROMPT (around line 1495 of trade_monitor.py). It walks six gates (G1..G6 plus G7 WHALE_PATTERN added recently) then asks the brain to synthesize HOLD/SELL/WATCH. Editing this prompt is the most direct way to change brain behaviour - just modify the string and redeploy. The ENVIRONMENT block at the top is templated with the source-aware values from the context dict so the brain knows whether to defend a +20% TP or moonbag a +100% TP.

Brain memory across cycles

Each provider has its own JSON memory file in solana/: claude_brain_memory.json, gemini_brain_memory.json, groq_brain_memory.json. Each entry is keyed by mint. The brain reads previous notes for that mint at the start of every cycle and writes new notes at the end. This is what lets a brain say 'price already bounced from here once - waiting for second confirmation' across multiple polling cycles.

Learning engine

File: packages/python/elizaos/plugins/solana/learning_engine.py

Outcomes log on AWS: /home/ubuntu/eliza/.../learning_outcomes.json

Pattern file consumed by brains: winning_patterns.json (5 rules currently injected into all brains).

On startup, bootstrap_from_copy_trades() seeds the learning_outcomes from paper trades.

11. Jarvis - Chat Command Centre

File: `packages/python/elizaos/plugins/solana/jarvis_command_centre.py`

Jarvis is a router that intercepts `/message` POSTs (BotChat in the dashboard) BEFORE the generic `runtime.send_message()` handler. It pattern-matches the user's text against operational commands (check positions, summarise trades, change a config knob, dump a wallet's recent activity, etc.) and answers directly. If no Jarvis command matches the message falls through to the generic LLM.

Persistence

- `jarvis_conv_history.json` - chat history.
- `jarvis_winning_configs.json` - configs that produced wins, restored on demand.

Editing Jarvis: open `jarvis_command_centre.py` and add a new branch in `route()`. Each command is a small async function that returns a string. The Jarvis context can call into `runtime.get_service()` to reach any registered service (PositionManager, MonsterStrategy, etc.) and into `live_config` to read or write knobs.

12. Common Workflows (Recipes)

Recipe: add a new direct creator wallet to creator_alpha

- Edit packages/python/elizaos/plugins/solana/creator_alpha_playbook.json.
- Add an object inside direct_creators.tier_a / tier_b / tier_c / high_hit. Required fields: wallet, creates, graduated, rate, best_mc. Optional: note.
- Validate: `python3 -c "import json; json.load(open('<path>'))"`.
- SCP only the JSON file to AWS.
- `sudo systemctl restart traderbot.service`.
- Tail `traderbot.out` and look for '[creator-alpha] polling N direct creators' to confirm the new count.

Recipe: change creator_alpha sizing (e.g. 0.10 -> 0.20 SOL)

- Open the dashboard at `http://63.180.211.88:3001` and use the PositionConfigPanel - it PATCHes `/api/config` and persists immediately, no restart needed. (Easiest path.)
- Or edit `live_config.py` default for `creator_alpha_size_sol`, delete `live_config.json` on the server, restart. (Persistent change to the default.)

Recipe: disable a strategy

- Top-of-file flag: edit `monster_signals.py` constants such as `LIFECYCLE_VIRAL_ENABLED = False`. Deploy + restart.
- Or set the corresponding `live_config` key (e.g. `monster_strategy_enabled=false`) via `PATCH /api/config`.

Recipe: edit the brain HOLD/SELL prompt

- Open `trade_monitor.py` around line 1495 (`_HOLD_SELL_PROMPT`).
- Mind the {placeholders} - they are substituted with the source-aware context values.
- Deploy + restart. New prompt takes effect on the NEXT brain cycle for every open position (no hot reload).

Recipe: add a new dashboard panel that shows new data

- In `dashboard_api.py`: add `async def handle_<thing>(request) -> web.Response`, then `app.router.add_get('/api/<thing>', handle_<thing>)` near the other adds.
- In `packages/dashboard/src/components/diagnostics/` create `<Thing>Panel.tsx` that fetches the new endpoint via `fetch('/api/<thing>')` in a `useEffect` with an interval.
- Import and place the panel inside `App.tsx` (or `AlphaDashboardLayout.tsx`).
- `cd packages/dashboard && bun run build`.
- SCP `packages/dashboard/dist/` AND the modified `dashboard_api.py`.
- `systemctl restart traderbot.service`.

Recipe: investigate a missed token (the MOG postmortem pattern)

- Find the mint address. Find the create signer via `getSignaturesForAddress` on the MINT, take the OLDEST sig, `getTransaction`, take `keys[0]` as the signer.
- If signer is a launchpad PDA (e.g. signed via `FLASHX8DrLbge...`), our scout will not have caught it: dev wallet is not the on-chain signer.
- Helper scripts go in `/tmp/` for ad-hoc investigation - they can use Helius RPC directly with the API key from `.env`. Do not commit `/tmp` scripts.
- Cross-check: when you extract a wallet from a script, ALWAYS print the FULL address and verify - many wallets share long prefixes and a 30-char truncation can hide divergence.

Recipe: emergency stop

- Dashboard: PositionConfigPanel has a kill toggle that PATCHes `monster_strategy_enabled=false` and `creator_alpha_max_concurrent=0`.
- SSH: `sudo systemctl stop traderbot.service`. The bot ungracefully halts; open positions remain open on-chain (no auto-flatten).
- To force-close a single position: `POST /api/positions/{mint}/close`.

13. Monitoring and Debugging

Logs

- Single combined log: /home/ubuntu/eliza/traderbot.out (stdout + stderr appended).
- Rotation: NOT configured. The file grows unbounded. Truncate with 'sudo truncate -s 0 traderbot.out' if it gets large.
- systemd journal: 'sudo journalctl -u traderbot.service -f' (mostly Type=notify and watchdog events).

Useful tail filters

```
# All creator-alpha activity
tail -f traderbot.out | grep -E '\[creator-alpha\]'

# Brain decisions on every open mint
tail -f traderbot.out | grep -E 'BRAIN|GROQ|GEMINI|CLAUDE'

# Errors and tracebacks
tail -f traderbot.out | grep -E 'ERROR|Traceback|exception'

# Buy / sell tx outcomes
tail -f traderbot.out | grep -E 'OPEN|CLOSE|TX:|sig=|HTTP 4|HTTP 5'
```

Health check endpoints

- GET /health -> {status:'ok', agent:'TraderBot'} (defined in run_traderbot.py).
- GET /api/status -> richer summary: balance, open positions, equity, scout status.
- Both are public on the EC2 instance's port 3001.

Common gotchas

- PumpPortal HTTP 200 + sig does NOT guarantee on-chain landing - parse logs / poll Helius.
- Jupiter lite.jupiterapi.com is unreachable from WSL2; use public.jupiterapi.com.
- DexScreener: pick HIGHEST LIQUIDITY pair, not h24 volume.
- Newly graduated bonding-curve tokens may not be on DexScreener for 30-90s - use BC entry price fallback (sol_size / tokens_received_raw / 1e6).
- PumpPortal sell amount: use UI tokens or '100%' string, NEVER raw lamports.
- PumpPortal pump-amm BUY needs wallet >= sol_size * (1 + slippage).

14. Known Limitations and Recent Findings

MOG / Axiom-launchpad blind spot (discovered 2026-05-02)

The creator_alpha scout ONLY detects tokens whose on-chain create signer is a wallet listed in the playbook AND where the tx contains pump.fun program 6EF8rrec... AND the log line 'Instruction: Create'.

When a token is launched via Axiom (FLASHX8DrLbge... program), Axiom creates an EPHEMERAL signer PDA per launch. The dev wallet (e.g. 4euyaqBe...) does not appear in the create tx at all. pump.fun program IS still invoked (inner instruction) and the mint still ends in '...pump', but _creator_alpha_detect_create() returns None because (a) the signer check fails and (b) the outer logs do not include 'Instruction: Create'. Result: every Axiom-launched monster is missed.

Three potential fixes (none yet implemented): subscribe to FLASHX program directly via LaserStream; watch dev wallets for pre-launch Axiom-session setup txs; or accept the gap and rely on the slower lifecycle/DexScreener path for Axiom-mediated launches.

Stale-create bug (fixed 2026-05-02)

Operator polling fired on a HISTORICAL pump.fun create from the funded wallet because we compared the create tx blockTime against the funding event. Fix: in the watched-children loop, skip any create sig whose blockTime is more than 60s BEFORE the funding event. Commit 37a48f14fe.

Stale playbook entries (cleanup 2026-05-02)

Earlier MOG investigation traced GJLsSBfVuP (a non-MOG-related wallet) and added a transcription-error operator address (3zd5av5f3hxVhhgCDTbge7vrsmXA69wkMdaUcNHcCCSe - 0 balance, 0 history, INVALID). Both have been removed. Lesson: when extracting addresses from script output, always print and verify the FULL 44-char string. Many wallets share 30-char prefixes.

Currently disabled paths (feature-flagged off)

- Copy-trade and Frost Mirror (permanently off 2026-04-24).
- lifecycle_viral (0/5 WR, disabled 2026-04-30).
- Strategy B/C/D in monster_signals - only D (Metora) intermittently active.
- Cluster-confirm scout, serial-deployer scout, breakout-candle scout.

Re-enabling: flip the constant in monster_signals.py, redeploy. The scout loop and gate code is still present.

15. Quick Reference Card

File you most commonly edit

Detection / scouts	packages/python/elizaos/plugins/solana/monster_signals.py
Position open / exits	packages/python/elizaos/plugins/solana/strategy_e_monster.py
Brain prompts / cascades	packages/python/elizaos/plugins/solana/trade_monitor.py
Wallet roster	packages/python/elizaos/plugins/solana/creator_alpha_playbook.json
Runtime config	packages/python/elizaos/plugins/solana/live_config.py + live_config.json
HTTP API routes	packages/python/elizaos/plugins/solana/dashboard_api.py
Dashboard panels	packages/dashboard/src/components/diagnostics/
Dashboard root	packages/dashboard/src/App.tsx
Bot entry	packages/python/run_traderbot.py
Plugin factory	packages/python/elizaos/plugins/solana/__init__.py

Commands you most commonly run

```
# Tail bot log on AWS
ssh -i /home/paulf/.ssh/traderbot-frankfurt-key.pem ubuntu@63.180.211.88 \
    "tail -f /home/ubuntu/eliza/traderbot.out"

# Restart bot
ssh -i /home/paulf/.ssh/traderbot-frankfurt-key.pem ubuntu@63.180.211.88 \
    "sudo systemctl restart traderbot.service"

# Deploy a Python file
scp -i /home/paulf/.ssh/traderbot-frankfurt-key.pem \
    packages/python/elizaos/plugins/solana/<file>.py \
    ubuntu@63.180.211.88:/home/ubuntu/eliza/packages/python/elizaos/plugins/solana/<file>.py

# Build dashboard
cd packages/dashboard && bun run build

# Validate JSON before deploy
python3 -c "import json; json.load(open('<file>.json'))"

# Local Python syntax check
/home/paulf/eliza/.venv_py/bin/python -c \
    "import ast; ast.parse(open('<file>.py').read())"
```

Endpoints quick list

```
GET    /api/status
GET    /api/positions
GET    /api/history?limit&offset
GET    /api/equity?since
GET    /api/launches?limit
GET    /api/creator-alpha
GET    /api/lifecycle/rejects
GET    /api/performance-by-source
GET    /api/brain-memory
PATCH /api/config
POST   /api/positions/{mint}/close
POST   /api/control
GET    /ws
```

Final reminder

All deployment is human-in-the-loop `scp` + `systemctl` restart. There is no auto-deploy from git push (the `deploy-dashboard.yml` workflow is present but not in the active path). Always verify '`sudo systemctl is-active traderbot.service`' returns 'active' after a restart. If the bot fails to start, tail `traderbot.out` for the traceback.